Univerzitet u Bihaću
Tehnički fakultet
Odsjek: Elektrotehnika
Smjer: Računarstvo i Informatika

Objektno orijentirane baze podataka

Projektni zadatak

Tema: LabInventory

Profesor: doc. dr. Admir Midžić
Asistent: MA Zinaid Kapić

Student:
Suad Kucalović, 1272

Bihać, februar 2026. godine

Sažetak

U savremenom razvoju informacionih sistema sve veći značaj ima izrada aplikacija koje omogućavaju efikasno upravljanje resursima i pouzdanu evidenciju podataka. U ovom radu razvijena je web aplikacija za upravljanje laboratorijskim inventarom pod nazivom *LabInventory*, koja omogućava digitalnu evidenciju opreme i materijala u laboratorijskom okruženju. Aplikacija je razvijena korištenjem Laravel frameworka, koji omogućava stabilan backend, sigurnu autentifikaciju korisnika i efikasno upravljanje podacima.

Sistem omogućava kreiranje, pregled i uređivanje laboratorijskih stavki, praćenje količina i minimalnih zaliha, upravljanje inventarskim zapisima, lokacijama i proizvođačima, kao i upravljanje korisnicima sa različitim nivoima pristupa. Podaci se pohranjuju u relacionu bazu podataka, dok je korisnički interfejs dizajniran tako da bude jednostavan, pregledan i prilagođen svakodnevnom radu laboratorijskog osoblja. Također je implementiran REST API koji omogućava programski pristup podacima o stavkama inventara.

Aplikacija predstavlja praktično rješenje za digitalizaciju upravljanja laboratorijskom opremom, smanjujući mogućnost grešaka i olakšavajući praćenje stanja inventara kroz moderan i funkcionalan sistem.

Ključne riječi: web aplikacija, laboratorijski inventar, Laravel, CRUD, REST API, baza podataka

Abstract

In modern information system development, building applications that enable efficient resource management and reliable data tracking has become increasingly important. This paper presents a web application for laboratory inventory management called *LabInventory*, designed to provide digital tracking of laboratory equipment and materials. The application is developed using the Laravel framework, which ensures a stable backend, secure user authentication, and efficient data management.

The system allows creation, viewing, and editing of inventory items, monitoring stock quantities and minimum thresholds, managing inventory records, locations, and manufacturers, as well as user management with role-based access control. Data is stored in a relational database, while the user interface is designed to be clean, intuitive, and suitable for everyday laboratory use. Additionally, a REST API is implemented to enable programmatic access to inventory data.

This application represents a practical solution for the digitalization of laboratory inventory management, reducing errors and improving transparency through a modern and functional web system.

Keywords: web application, laboratory inventory, Laravel, CRUD, REST API, database

Sadržaj

1 Uvod **1**

2 Modeliranje aplikacije **2**
2.1 Opis aplikacije 2
2.2 Statički UML dijagrami 3
2.2.1 Klasni dijagram 4
2.3 Dinamički UML dijagrami 5
2.3.1 Dijagrami slučajeva korištenja 5
2.3.2 Sekvencijalni dijagrami 7
2.4 ER dijagram baze podataka 10

3 Implementacija **12**
3.1 Tehnologija izrade aplikacije 12
3.2 Laravel 12
3.3 MySQL 13
3.4 MVC arhitektura 13
3.5 Objektno-relaciono mapiranje 14
3.6 REST Api 15

4 Analiza rada aplikacije **16**
4.1 Opis slučajeva korištenja 16
4.2 Testiranje API-ja 27

5 Zaključak **29**

Popis slika

1 Klasni dijagram aplikacije LabInventory 4
2 Dijagram slučajeva korištenja 6
3 Sekvencijalni dijagram za prijavu korisnika 7
4 Sekvencijalni dijagram za kreiranje stavke 8
5 Sekvencijalni dijagram za pregled detalja stavke 9
6 Entity Relationship dijagram 10
7 MVC arhitektura 14
8 Prijava korisnika na sistem 16
9 Prijava korisnika na sistem netačna 17
10 Prikaz Dashboard-a (stranica sa svim relevantnim informacijama) 18
11 Odjava korisnika sa sistema 19
12 Pregled Dashboard stranice 20
13 Pregled liste stavki 21
14 Pregled liste korisnika 22
15 Dodavanje novog korisnika 23
16 Poruka o uspješnom dodavanju korisnika 23
17 Ažuriranje korisnika 24
18 Poruka o uspješnom ažuriranju korisničkih podataka 25
19 Brisanje korisnika 26
20 Poruka o uspješnom brisanju korisnika 26
21 Testiranje Items API rute u alatu Postman 28

1 Uvod

Projekat koji je razvijen predstavlja web aplikaciju za upravljanje laboratorijskim inventarom, s ciljem unapređenja evidencije opreme i materijala u obrazovnim i istraživačkim institucijama. U savremenom laboratorijskom okruženju, vođenje precizne evidencije o stanju opreme, količini potrošnog materijala i inventarskim zapisima predstavlja važan segment organizacije rada. Tradicionalni načini evidencije, zasnovani na papirnoj dokumentaciji ili nepovezanim tabelama, često dovode do grešaka, netačnih podataka i otežanog praćenja promjena. Zbog toga je postojala potreba za razvojem aplikacije koja će omogućiti centralizovano, sigurno i pregledno upravljanje laboratorijskim resursima.

Aplikacija je razvijena korištenjem Laravel PHP frameworka, koji omogućava izradu stabilnih, sigurnih i skalabilnih web aplikacija. Laravel se zasniva na MVC (Model–View–Controller) arhitekturi, čime se postiže jasna podjela odgovornosti unutar sistema. Modeli su zaduženi za komunikaciju sa bazom podataka i upravljanje podacima, kontroleri obrađuju poslovnu logiku aplikacije, dok su pogledi (views) odgovorni za prikaz korisničkog interfejsa. Ovakav pristup omogućava lakše održavanje sistema, jednostavniju nadogradnju funkcionalnosti i efikasnije testiranje aplikacije.

Aplikacija omogućava korisnicima pregled svih laboratorijskih stavki, pregled detalja pojedinačne opreme ili materijala, kao i upravljanje inventarskim zapisima. Administrator ima mogućnost upravljanja korisnicima, kategorijama, lokacijama, proizvođačima i mjernim jedinicama, dok laborant može kreirati i uređivati stavke inventara. Sistem također vodi evidenciju aktivnosti kroz log zapise, čime se omogućava praćenje izmjena i povećava transparentnost rada.

Za pohranu podataka koristi se relaciona baza podataka, koja omogućava sigurno i efikasno upravljanje informacijama o korisnicima, stavkama, inventarskim zapisima i pratećim entitetima. Korištenjem Eloquent ORM-a, Laravel omogućava rad sa bazom podataka kroz objektno orijentisan pristup, čime se pojednostavljuje implementacija relacija i smanjuje potreba za pisanjem kompleksnih SQL upita.

Aplikacija podržava osnovne sigurnosne mehanizme, uključujući autentifikaciju korisnika, kontrolu pristupa na osnovu korisničkih uloga i validaciju unosa podataka. Time se osigurava da samo ovlašteni korisnici mogu pristupati određenim funkcionalnostima sistema, poput upravljanja inventarom ili administracije korisnika.

Cilj projekta je izrada funkcionalne, pregledne i sigurne web aplikacije koja pojednostavljuje proces upravljanja laboratorijskom opremom i omogućava pouzdano praćenje stanja inventara. Korištenjem savremenih tehnologija i principa razvoja softvera, aplikacija pruža stabilno i skalabilno rješenje koje se može dalje nadograđivati i prilagođavati potrebama laboratorijskog okruženja.

2 Modeliranje aplikacije

Modeliranje aplikacije predstavlja jedan od ključnih koraka u procesu razvoja softverskog sistema, jer omogućava jasno definisanje strukture aplikacije, njenih funkcionalnosti i međusobnih odnosa između pojedinih komponenti. Prije same implementacije bilo je neophodno analizirati potrebe laboratorijskog okruženja, definisati osnovne funkcionalnosti sistema te predvidjeti način na koji će različiti tipovi korisnika komunicirati s aplikacijom. Ovakav pristup omogućava stabilan razvoj, bolju organizaciju koda i lakše održavanje sistema.

U okviru projekta *LabInventory*, modeliranje aplikacije obuhvata izradu UML dijagrama koji prikazuju strukturu sistema, interakciju između korisnika i aplikacije, kao i organizaciju baze podataka. Korišteni su dijagrami slučajeva upotrebe (Use Case), dijagrami klasa i sekvencijski dijagrami, koji zajedno pružaju jasan uvid u funkcionisanje aplikacije. Ovi dijagrami opisuju ključne funkcionalnosti poput autentifikacije korisnika, upravljanja laboratorijskim stavkama, praćenja inventarskih zapisa, upravljanja korisnicima i evidencije aktivnosti putem log sistema.

Posebna pažnja posvećena je modeliranju relacija između entiteta kao što su stavke, kategorije, lokacije, proizvođači, inventarski zapisi i korisnici, kako bi se osigurala konzistentnost i integritet podataka. Također je definisana hijerarhija korisničkih uloga (administrator, laborant i read-only korisnik), čime su precizno određena prava pristupa i dozvoljene akcije unutar sistema.

Modeliranje sistema omogućava lakše planiranje razvoja aplikacije, smanjuje mogućnost grešaka tokom implementacije i olakšava buduća proširenja funkcionalnosti. Također predstavlja važan dio tehničke dokumentacije koja pomaže u razumijevanju strukture i logike sistema svim učesnicima u razvoju. Na ovaj način postavljeni su čvrsti temelji za izgradnju stabilne, funkcionalne i skalabilne web aplikacije namijenjene upravljanju laboratorijskim inventarom.

2.1 Opis aplikacije

Aplikacija LabInventory predstavlja web sistem namijenjen upravljanju laboratorijskom opremom, materijalima i inventarom u okviru obrazovne ili istraživačke institucije. Cilj aplikacije je omogućiti efikasno vođenje evidencije o laboratorijskim resursima, praćenje njihove dostupnosti, količine i statusa, kao i olakšati administrativne procese vezane za upravljanje opremom. Sistem omogućava centralizovano upravljanje podacima o laboratorijskim uređajima, potrošnim materijalima i drugim resursima. Svaki inventarski element sadrži detaljne informacije poput naziva, serijskog broja, kategorije, lokacije, količine, datuma nabavke i trenutnog statusa (dostupan, u upotrebi, neispravan i slično). Na taj način osigurava se precizna evidencija i bolja organizacija laboratorijskih sredstava.

Aplikacija razlikuje više tipova korisnika, poput administratora i standardnih korisnika (npr. asistenata ili laboratorijskog osoblja). Administrator ima proširene privilegije koje uključuju dodavanje, uređivanje i brisanje inventarskih stavki, upravljanje korisnicima sistema te pregled kompletne evidencije.

Standardni korisnici mogu pregledavati dostupnu opremu, evidentirati korištenje određenih resursa ili podnositi zahtjeve za korištenje opreme.

Sistem je osmišljen tako da minimizira mogućnost grešaka u vođenju evidencije i smanji rizik od gubitka ili neadekvatnog korištenja laboratorijske opreme. Automatska validacija unosa podataka, kontrola pristupa i jasna struktura baze podataka doprinose pouzdanosti i sigurnosti sistema. LabInventory aplikacija pruža pregledan i responzivan korisnički interfejs, prilagođen radu na različitim uređajima. Korištenjem savremenih tehnologija i principa razvoja softvera, sistem omogućava jednostavno održavanje, proširenje funkcionalnosti i integraciju sa drugim informacionim sistemima u budućnosti.

2.2 Statički UML dijagrami

Statički UML dijagrami predstavljaju način vizualnog prikaza strukture softverskog sistema i njegovih osnovnih elemenata. Ovi dijagrami koriste se za opis arhitekture aplikacije, odnosa između komponenti i organizacije podataka unutar sistema. Njihova glavna svrha je olakšavanje razumijevanja dizajna aplikacije prije same implementacije, kao i pojednostavljivanje održavanja i daljeg razvoja sistema. U razvoju softverskih rješenja, UML dijagrami omogućavaju inženjerima da jasno predstave strukturu aplikacije, tokove podataka i međusobne zavisnosti između komponenti. Statički dijagrami fokusirani su na prikaz trajnih elemenata sistema, bez obzira na njihovo ponašanje tokom izvršavanja. U okviru ovog projekta korišteni su sljedeći tipovi statičkih UML dijagrama:

- Dijagram strukture sistema – prikazuje osnovne dijelove aplikacije i njihov međusobni odnos.
- Klasni dijagram – definiše klase, njihove atribute, metode i veze između njih.
- Objektni dijagram – prikazuje konkretne instance klasa u određenom trenutku.
- Dijagram kompozicije – prikazuje odnose zavisnosti i strukturu složenih komponenti sistema.
- Dijagram ograničenja – opisuje pravila i ograničenja koja važe unutar sistema.

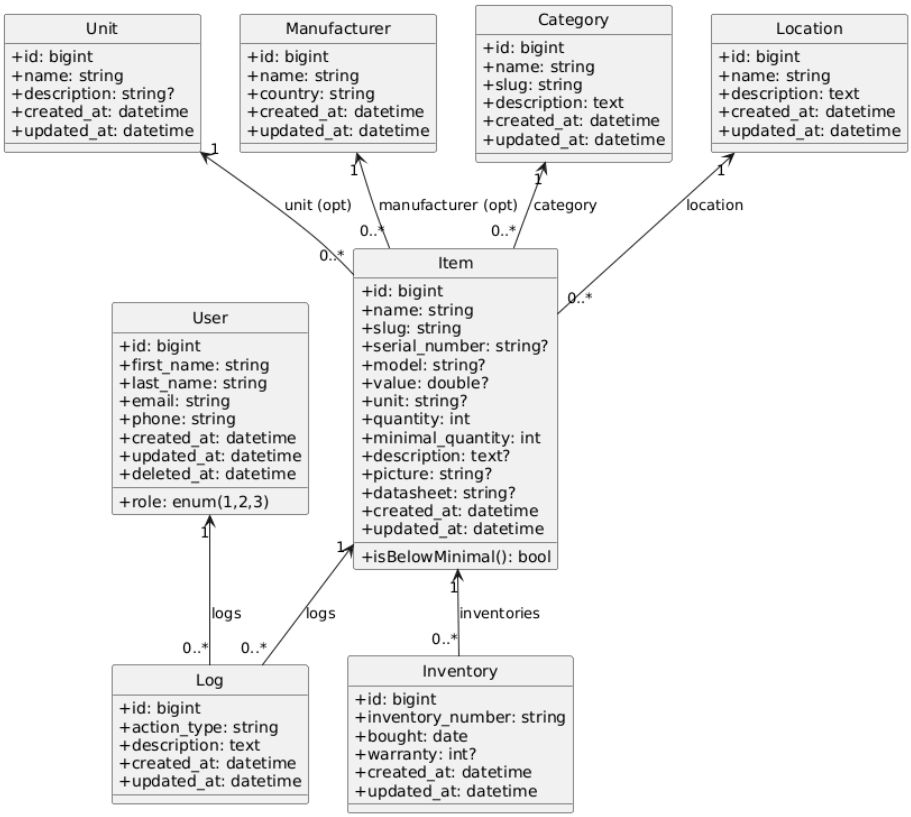
U nastavku rada detaljnije je prikazan i objašnjen klasni dijagram, koji predstavlja osnovu za razumijevanje strukture aplikacije i njenih ključnih funkcionalnosti.

2.2.1 Klasni dijagram

Klasni dijagram prikazuje osnovnu strukturu aplikacije *LabInventory* kroz glavne entitete i njihove međusobne odnose. U sistemu se koriste klase *User*, *Item*, *Category*, *Location*, *Manufacturer*, *Unit*, *Inventory* i *Log*, koje zajedno omogućavaju upravljanje laboratorijskim inventarom i evidencijom aktivnosti u sistemu.

Klasa *Item* predstavlja centralni entitet sistema i povezana je sa klasama *Category* i *Location*, pri čemu svaka stavka pripada jednoj kategoriji i jednoj lokaciji. Također, stavka može imati definisanog proizvođača (*Manufacturer*) i mjernu jedinicu (*Unit*), ali te veze mogu biti opcionalne.

Jedna stavka može imati više inventarskih zapisa (*Inventory*), dok svaki inventarski zapis pripada tačno jednoj stavci. Pored toga, sistem vodi evidenciju aktivnosti putem klase *Log*, gdje jedan korisnik (*User*) može imati više log zapisa, kao i jedna stavka. Ovakva struktura omogućava jednostavno održavanje i proširenje sistema.
Klasni dijagram aplikacije prikazan je na slici 1.



Slika 1: Klasni dijagram aplikacije LabInventory

2.3 Dinamički UML dijagrami

Dinamički UML dijagrami opisuju ponašanje sistema tokom vremena i prikazuju način na koji se njegove komponente međusobno povezuju i reaguju na određene događaje. Ovi dijagrami omogućavaju razumijevanje toka izvršavanja procesa, razmjene podataka i interakcije između korisnika i sistema. Posebno su značajni za prikaz realnog ponašanja aplikacije tokom izvršavanja pojedinih funkcionalnosti. Dinamički dijagrami se koriste za modeliranje vremenskih sekvenci, promjena stanja sistema, kao i komunikacije između različitih komponenti. Na taj način omogućavaju detaljniji uvid u rad aplikacije i pomažu u identifikaciji potencijalnih problema tokom implementacije. U okviru UML standarda razlikuje se nekoliko vrsta dinamičkih dijagrama:

- Dijagram aktivnosti,
- Dijagram stanja,
- Dijagram slučajeva korištenja,
- Sekvencijalni dijagram,
- Komunikacioni dijagram.

U nastavku rada detaljnije su opisani i prikazani dijagram slučajeva korištenja i sekvencijalni dijagram, jer oni najjasnije predstavljaju interakciju korisnika sa sistemom i tok izvršavanja osnovnih funkcionalnosti aplikacije.

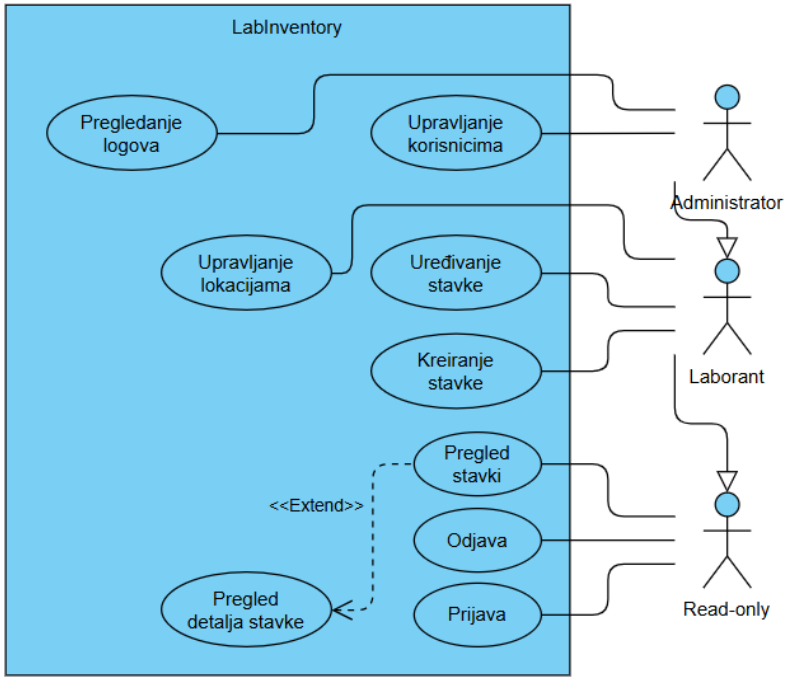
2.3.1 Dijagrami slučajeva korištenja

Dijagrami slučajeva korištenja (engl. *Use Case Diagram*) koriste se za prikaz funkcionalnosti sistema iz perspektive krajnjih korisnika i njihove interakcije sa aplikacijom. U okviru projekta *LabInventory*, ovi dijagrami omogućavaju jasan pregled načina na koji različiti tipovi korisnika koriste sistem i koje radnje mogu izvršavati unutar aplikacije.

Use case dijagrami se koriste za ilustraciju osnovnih funkcionalnih zahtjeva sistema, bez ulaska u tehničke detalje implementacije. Njihova svrha je da prikažu scenarije korištenja sistema, uloge korisnika i granice sistema, čime se olakšava razumijevanje ponašanja aplikacije u realnim uslovima.

U aplikaciji *LabInventory* definisana su tri tipa aktera: *Read-only* korisnik, *Laborant* i *Administrator*. Read-only korisnik ima mogućnost prijave na sistem, pregleda stavki, pregleda detalja pojedinačne stavke, pregleda inventarskih zapisa i pregleda logova aktivnosti. Laborant, pored navedenih funkcionalnosti, može kreirati i uređivati stavke te upravljati inventarskim zapisima. Administrator ima najviši nivo privilegija i, pored svih prethodnih mogućnosti, može upravljati korisnicima sistema, lokacijama, kao i drugim pomoćnim podacima.

Prikazani dijagram slučajeva korištenja razvijene aplikacije prikazan je na slici 2.



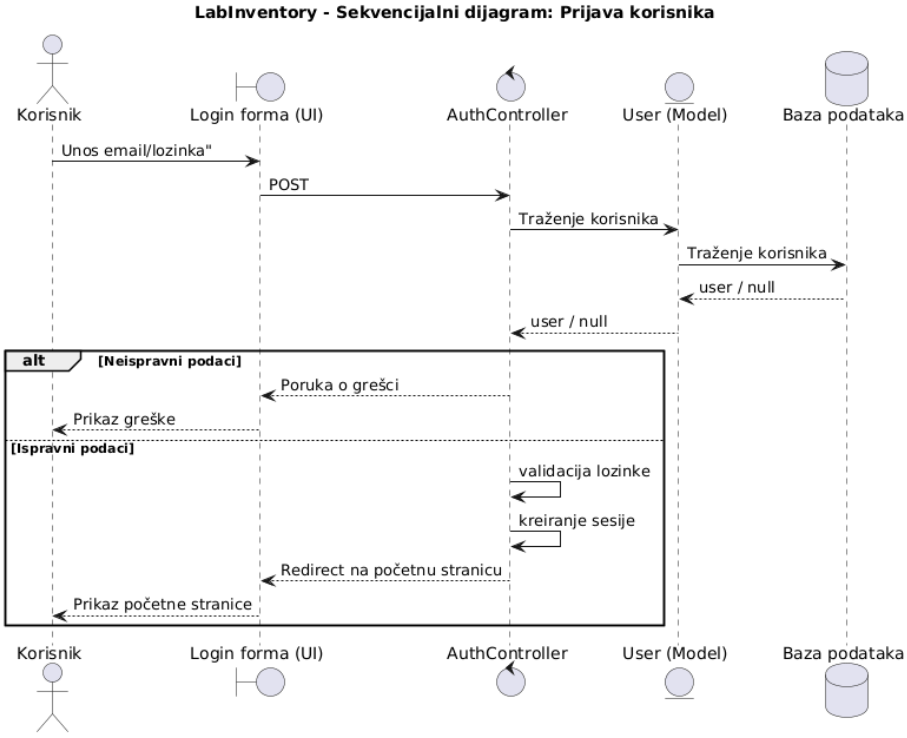
Slika 2: Dijagram slučajeva korištenja

Dijagram slučajeva korištenja pruža pregled ključnih funkcionalnosti aplikacije i predstavlja osnovu za dalju analizu sistema, izradu ostalih UML dijagrama i implementaciju poslovne logike aplikacije.

2.3.2 Sekvencijalni dijagrami

Sekvencijalni dijagram koristi se za prikaz vremenskog slijeda interakcija između aktera i sistema, odnosno načina na koji se razmjenjuju poruke tokom izvršavanja određene funkcionalnosti. Ovaj dijagram omogućava jasno razumijevanje redoslijeda aktivnosti i komunikacije između korisnika, kontrolera, modela i baze podataka, bez ulaska u tehničke detalje implementacije.

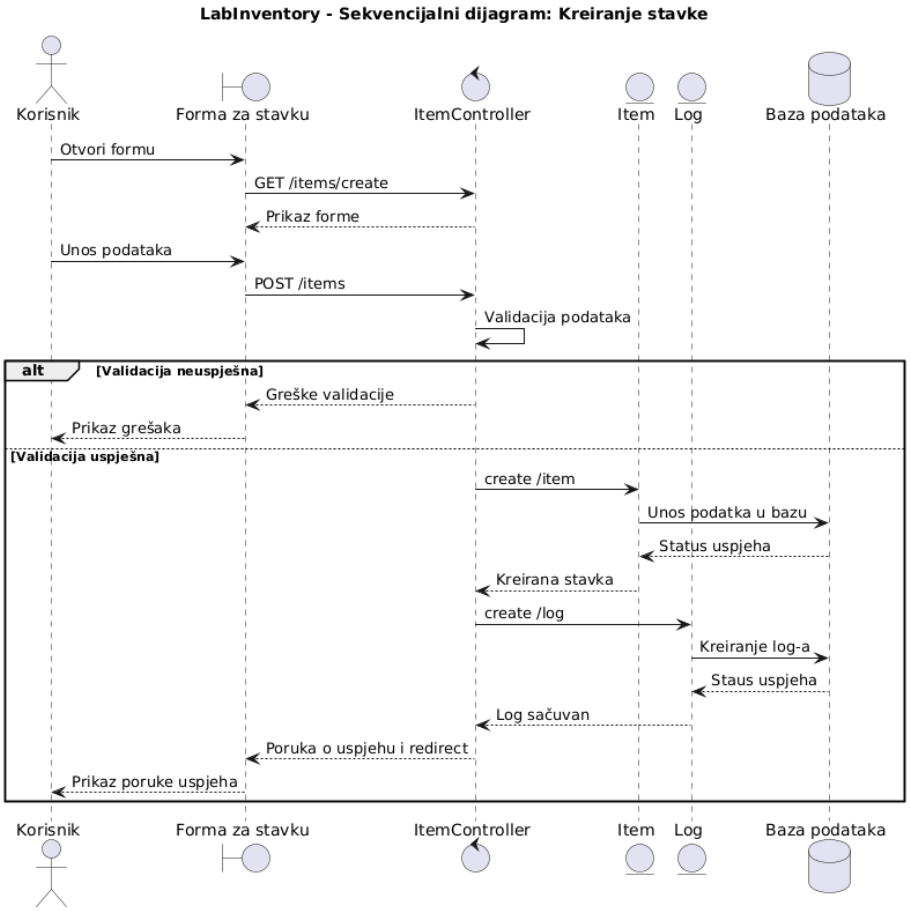
Na slici 3 prikazan je sekvencijalni dijagram za prijavu korisnika. Korisnik unosi svoje pristupne podatke putem login forme, nakon čega se zahtjev prosljeđuje *AuthController*-u. Kontroler vrši provjeru podataka u bazi putem modela *User*. U slučaju neispravnih podataka sistem vraća poruku o grešci, dok se u slučaju uspješne autentifikacije kreira sesija i korisnik se preusmjerava na početnu stranicu aplikacije.



Slika 3: Sekvencijalni dijagram za prijavu korisnika

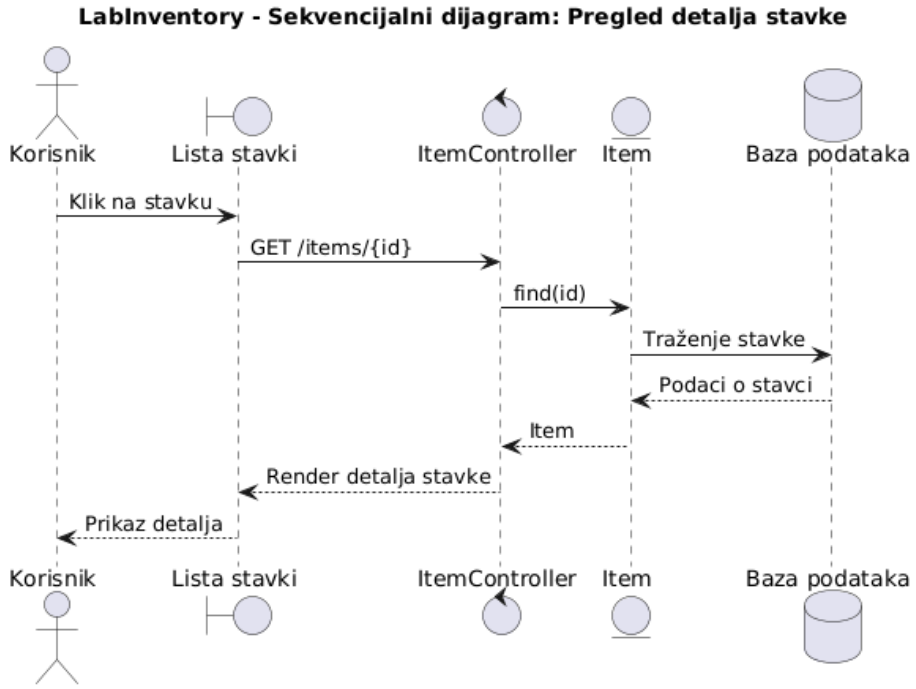
Na slici 4 prikazan je sekvencijalni dijagram za kreiranje nove stavke od strane laboranta ili administratora. Korisnik putem forme unosi podatke o stavci, nakon čega *ItemController* vrši validaciju unesenih podataka. Ukoliko validacija nije uspješna, sistem vraća odgovarajuće poruke o greškama. U slučaju uspješne validacije, podaci se pohranjuju u bazu podataka putem modela *Item*,

a zatim se evidentira odgovarajući log zapis putem modela *Log*. Nakon uspješnog izvršavanja operacije, korisniku se prikazuje poruka o uspjehu i vrši se preusmjeravanje.



Slika 4: Sekvencijalni dijagram za kreiranje stavke

Na slici 7 prikazan je sekvencijalni dijagram za pregled detalja stavke. Korisnik odabirom određene stavke pokreće zahtjev prema *ItemController*-u, koji putem modela *Item* dohvaća tražene podatke iz baze podataka. Nakon uspješnog preuzimanja podataka, sistem renderuje stranicu sa detaljima stavke i prikazuje je korisniku.



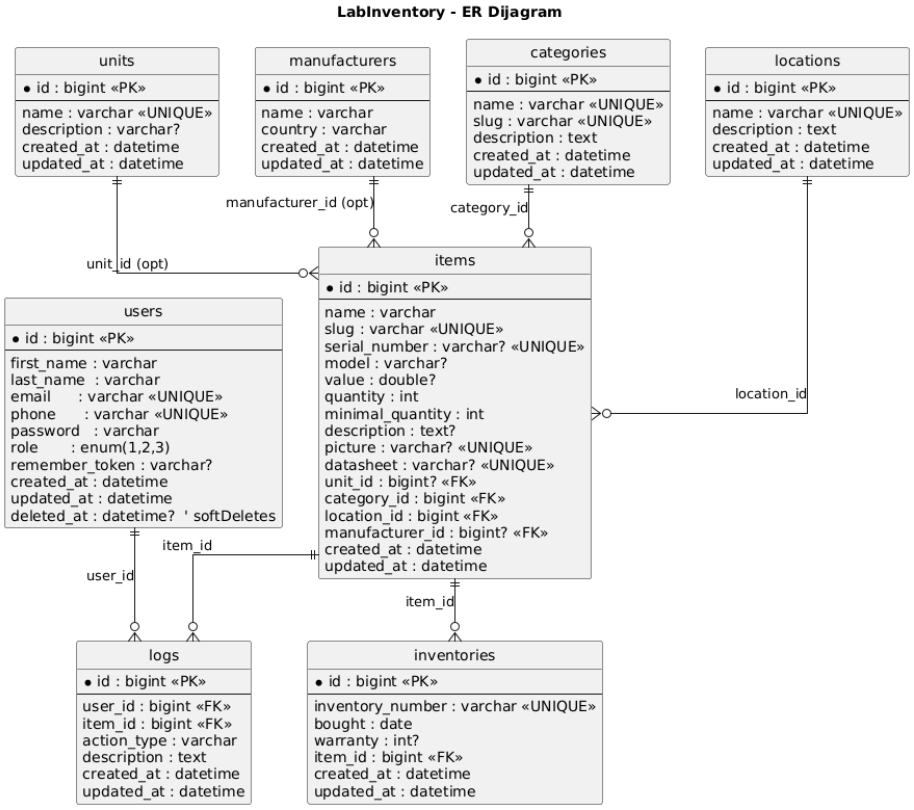
Slika 5: Sekvencijalni dijagram za pregled detalja stavke

Prikazani sekvencijalni dijagrami ilustruju način na koji sistem obrađuje korisničke zahtjeve, redoslijed izvršavanja operacija i komunikaciju između slojeva aplikacije. Ovi dijagrami doprinose boljem razumijevanju dinamike sistema i predstavljaju važan dio tehničke dokumentacije aplikacije *LabInventory*.

2.4 ER dijagram baze podataka

Baza podataka aplikacije *LabInventory* opisana je pomoću ER (Entity Relationship) dijagrama, koji prikazuje osnovne entitete i njihove međusobne veze. Sistem se sastoji od ključnih tabela: *users*, *items*, *inventories*, *logs*, *categories*, *locations*, *manufacturers* i *units*. Centralni entitet u bazi je tabela *items*, koja sadrži informacije o laboratorijskim stavkama (oprema ili materijal), uključujući naziv, količinu, minimalnu količinu, opis, te povezane podatke o kategoriji, lokaciji, proizvođaču i mjernoj jedinici.

Entitet *categories* služi za klasifikaciju stavki po kategorijama, dok entitet *locations* definiše fizičku lokaciju stavke. Entitet *manufacturers* omogućava evidenciju proizvođača, a entitet *units* definiše mjerne jedinice koje se mogu vezati za stavke. Tabela *inventories* predstavlja inventarske zapise pojedinačnih stavki, gdje se čuvaju podaci kao što su inventarski broj, datum kupovine i trajanje garancije. Tabela *logs* služi za evidenciju aktivnosti u sistemu, pri čemu svaki log zapis povezuje korisnika iz tabele *users* i stavku iz tabele *items*, uz dodatne informacije o tipu akcije i opisu.



Slika 6: Entity Relationship dijagram

Veze između entiteta su uglavnom tipa 1:N. Jedna kategorija može imati više stavki, kao i jedna lokacija, proizvođač ili mjerna jedinica, dok svaka stavka pripada tačno jednoj kategoriji i jednoj lokaciji (a proizvođač i jedinica mogu biti opcionalni). Jedna stavka može imati više inventarskih zapisa u tabeli *inventories*. Također, jedan korisnik može imati više zapisa u tabeli *logs*, kao i jedna stavka, što omogućava praćenje aktivnosti i promjena u sistemu. Ovakva struktura omogućava jednostavno upravljanje podacima, jasno definisane odnose između osnovnih elemenata sistema i pouzdanu evidenciju laboratorijskog inventara.

3 Implementacija

3.1 Tehnologija izrade aplikacije

Aplikacija je razvijena korištenjem programskog jezika PHP uz primjenu Laravel frameworka, koji predstavlja jedan od najpopularnijih alata za razvoj modernih web aplikacija. Laravel je zasnovan na MVC (engl. Model–View–Controller) arhitekturi, koja omogućava jasno razdvajanje poslovne logike, korisničkog interfejsa i upravljanja podacima, čime se postiže bolja organizacija koda i lakše održavanje sistema.

Korištenjem ovog arhitektonskog obrasca omogućena je veća preglednost aplikacije, jednostavnije testiranje i lakša nadogradnja funkcionalnosti. Modeli su zaduženi za rad s podacima i komunikaciju s bazom, kontroleri obrađuju korisničke zahtjeve, dok su prikazi odgovorni za vizuelnu prezentaciju podataka korisniku.

Za upravljanje bazom podataka korišten je MySQL sistem za upravljanje bazama podataka (DBMS), koji omogućava pouzdano, efikasno i sigurno čuvanje podataka. MySQL pruža stabilno okruženje za rad sa relacionim bazama, što je posebno pogodno za aplikacije koje upravljaju većim brojem povezanih entiteta, kao što su korisnici, stavke inventara, inventarski zapisi i logovi aktivnosti.

Kombinacijom Laravel frameworka i MySQL baze podataka osigurana je stabilna, skalabilna i efikasna osnova za razvoj web aplikacije *LabInventory*.

3.2 Laravel

Laravel je open-source PHP framework namijenjen razvoju modernih i skalabilnih web aplikacija. Odlikuje se jednostavnom sintaksom, jasno definisanom strukturom i bogatim skupom ugrađenih funkcionalnosti koje značajno ubrzavaju proces razvoja. Jedna od njegovih ključnih prednosti je implementacija objektnog relacionog mapiranja (ORM) putem Eloquent ORM-a, koji omogućava intuitivnu i sigurnu komunikaciju sa bazom podataka.

Laravel pruža napredne mehanizme za upravljanje rutama, autentifikacijom, autorizacijom, validacijom podataka i sigurnošću aplikacije. Također nudi podršku za izradu modularnih komponenti i korištenje Blade templating sistema, koji omogućava jednostavno kreiranje dinamičkih korisničkih interfejsa.

Zahvaljujući svojoj fleksibilnosti i bogatoj dokumentaciji, Laravel predstavlja pouzdan izbor za razvoj savremenih web aplikacija, omogućavajući bržu implementaciju funkcionalnosti, lakše održavanje koda i visok nivo sigurnosti aplikacije.

3.3 MySQL

MySQL je relacijski sistem za upravljanje bazama podataka koji omogućava efikasno čuvanje, obradu i upravljanje podacima. Podaci su organizovani u tabele koje se sastoje od redova i kolona, pri čemu svaka tabela predstavlja određeni entitet unutar sistema.

Zahvaljujući svojoj pouzdanosti, brzini i širokoj primjeni, MySQL je jedan od najčešće korištenih sistema za upravljanje bazama podataka u web aplikacijama.

U okviru projekta *LabInventory*, MySQL se koristi za pohranu podataka o korisnicima, laboratorijskim stavkama, inventarskim zapisima, kategorijama, lokacijama, proizvođačima, mjernim jedinicama i evidenciji aktivnosti, omogućavajući sigurno i efikasno upravljanje podacima.

3.4 MVC arhitektura

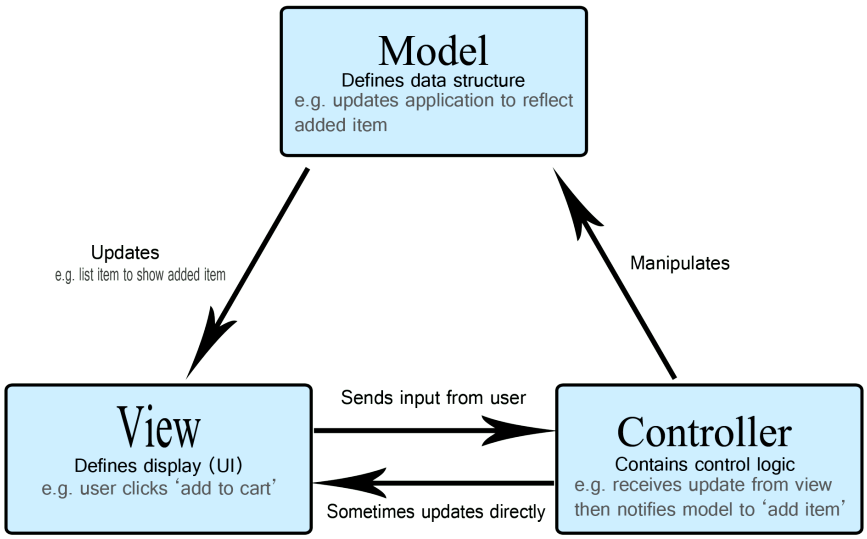
Model–View–Controller (MVC) predstavlja arhitektonski obrazac koji se koristi u razvoju softverskih aplikacija s ciljem jasnog razdvajanja odgovornosti unutar sistema. Ovaj pristup omogućava bolju organizaciju koda, lakše održavanje i jednostavnije proširivanje funkcionalnosti aplikacije.

Model predstavlja sloj zadužen za obradu podataka i poslovnu logiku aplikacije. On upravlja komunikacijom sa bazom podataka, definiše pravila rada sistema i omogućava pristup podacima kroz jasno definisane metode. U kontekstu aplikacije *LabInventory*, model obuhvata entitete kao što su korisnici, stavke (items), inventarski zapisi, kategorije, lokacije, proizvođači, mjerne jedinice i log zapisi.

View (pogled) je zadužen za prikaz podataka korisniku i predstavlja korisnički interfejs aplikacije. Ovaj sloj ne sadrži poslovnu logiku, već isključivo prikazuje informacije dobijene od modela, koristeći različite vizuelne komponente poput formi, tabela i listi. Jedan model može imati više različitih pogleda, što omogućava fleksibilan i prilagodljiv prikaz sadržaja.

Controller (kontroler) predstavlja vezu između modela i pogleda. Njegova uloga je da obrađuje korisničke zahtjeve, poziva odgovarajuće metode modela i određuje koji će se pogled prikazati korisniku. Kontroler time upravlja tokom aplikacije i omogućava pravilnu komunikaciju između svih komponenti sistema.

Primjena MVC arhitekture donosi brojne prednosti, uključujući lakše testiranje, bolju organizaciju koda i jednostavnije održavanje aplikacije. Zbog svoje fleksibilnosti i skalabilnosti, MVC obrazac je široko prihvaćen u modernim web okvirima kao što su Laravel, Ruby on Rails i ASP.NET, te predstavlja pouzdanu osnovu za razvoj kompleksnih web aplikacija poput sistema *LabInventory*.



Slika 7: MVC arhitektura

Kao što se može vidjeti na slici 5, prikazana je MVC arhitektura te komunikacija između komponenti, tačnije modela, pogleda i kontrolera.

3.5 Objektno-relaciono mapiranje

Objektno-relaciono mapiranje (ORM – Object Relational Mapping) predstavlja mehanizam koji omogućava rad s bazom podataka kroz objekte i klase unutar aplikacije, bez potrebe za direktnim pisanjem SQL upita. ORM omogućava povezivanje objekata iz programskog jezika sa tabelama u bazi podataka, čime se pojednostavljuje upravljanje podacima i unapređuje čitljivost i održivost koda.

Korištenjem ORM-a, programeri mogu izvršavati operacije poput dodavanja, ažuriranja, brisanja i čitanja podataka pomoću metoda i objekata, dok se generisanje SQL upita odvija automatski u pozadini. Na taj način olakšava se rad s bazom podataka i smanjuje mogućnost grešaka prilikom ručnog pisanja upita. ORM takođe omogućava lakše testiranje poslovne logike, jer aplikacija nije direktno vezana za konkretnu implementaciju baze podataka.

Dodatna prednost ORM pristupa jeste mogućnost jednostavne promjene sistema za upravljanje bazom podataka, na primjer prelazak sa MySQL-a na PostgreSQL, bez potrebe za značajnim izmjenama u aplikacijskom kodu. ORM djeluje kao posrednik između aplikacije i baze.

Programeri na taj način mogu raditi s podacima na apstraktnom nivou, bez direktnog upravljanja složenim SQL strukturama. U okviru ovog projekta korišten je ORM koji je sastavni dio Laravel frameworka – Eloquent ORM. Eloquent omogućava jednostavno mapiranje tabela baze podataka u modele, definisanje relacija između entiteta i efikasno upravljanje podacima, čime značajno doprinosi brzini razvoja, čitljivosti koda i stabilnosti aplikacije.

3.6 REST Api

REST API (Representational State Transfer – Application Programming Interface) predstavlja arhitektonski stil za izgradnju web servisa koji omogućava komunikaciju između klijenta i servera korištenjem standardnih HTTP metoda. Ovaj pristup zasniva se na principima jednostavnosti, skalabilnosti i bezstanjskog (stateless) rada, pri čemu svaki resurs unutar sistema, poput korisnika, stavki inventara ili inventarskih zapisa, ima jedinstveni identifikator (URI – Uniform Resource Identifier).

Klijent komunicira sa serverom koristeći standardne HTTP metode kao što su GET, POST, PUT i DELETE, koje omogućavaju dohvat, kreiranje, ažuriranje i brisanje podataka. Razmjena podataka se najčešće vrši u JSON formatu, što omogućava jednostavnu obradu, čitljivost i kompatibilnost sa različitim platformama i tehnologijama.

REST API omogućava modularan i fleksibilan dizajn sistema, gdje su funkcionalnosti jasno razdvojene i lako proširive. Ovakav pristup olakšava održavanje aplikacije, poboljšava njenu skalabilnost i omogućava jednostavnu integraciju sa drugim servisima ili klijentskim aplikacijama, uključujući web i mobilne platforme.

Zbog svoje jednostavnosti, pouzdanosti i široke primjene, REST API predstavlja standardno rješenje u razvoju savremenih web aplikacija, uključujući i aplikaciju *LabInventory*, gdje omogućava efikasnu komunikaciju između korisničkog interfejsa i serverske logike.

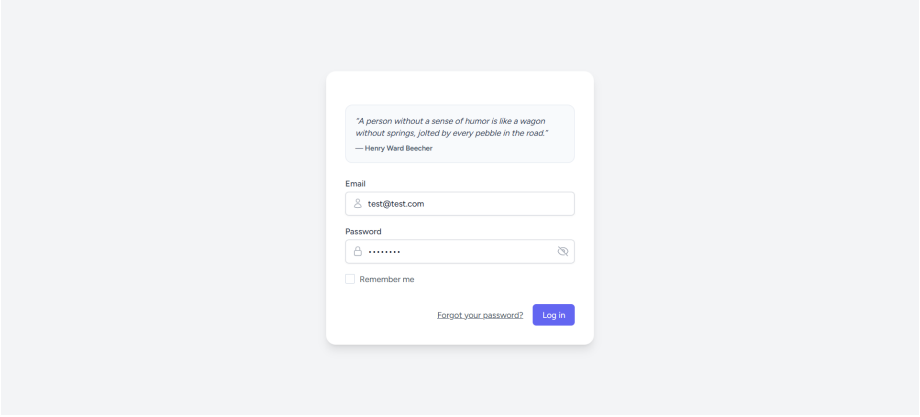
4 Analiza rada aplikacije

Analiza rada web aplikacije LabInventory izvršena je kroz definisanje ključnih slučajeva korištenja sistema. Za svaki slučaj identificirani su akteri, definisani preduslovi, osnovni tok izvršavanja te eventualni sporedni scenariji u slučaju greške.

4.1 Opis slučajeva korištenja

Slučaj korištenja 1: Prijava na sistem

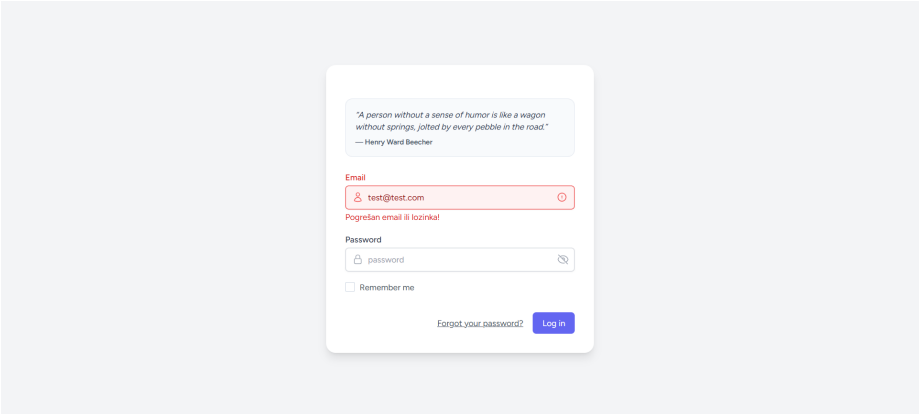
- Naziv SK: Prijava na sistem,
- Akteri SK: Registrovani korisnik,
- Učesnici SK: Korisnik i sistem,
- Preduslov: Korisnik posjeduje validan korisnički račun,
- Osnovni scenarij SK:
  - Korisnik unosi e-mail adresu i lozinku,
  - Korisnik klikne na dugme *Log in*,
  - Sistem provjerava ispravnost unesenih podataka,
  - Sistem autentificira korisnika,
  - Sistem preusmjerava korisnika na Dashboard stranicu.



Slika 8: Prijava korisnika na sistem

- Sporedni scenarij SK:

- Uneseni podaci nisu tačni,
- Sistem odbija prijavu,
- Prikazuje se poruka o grešci (neispravni kredencijali).

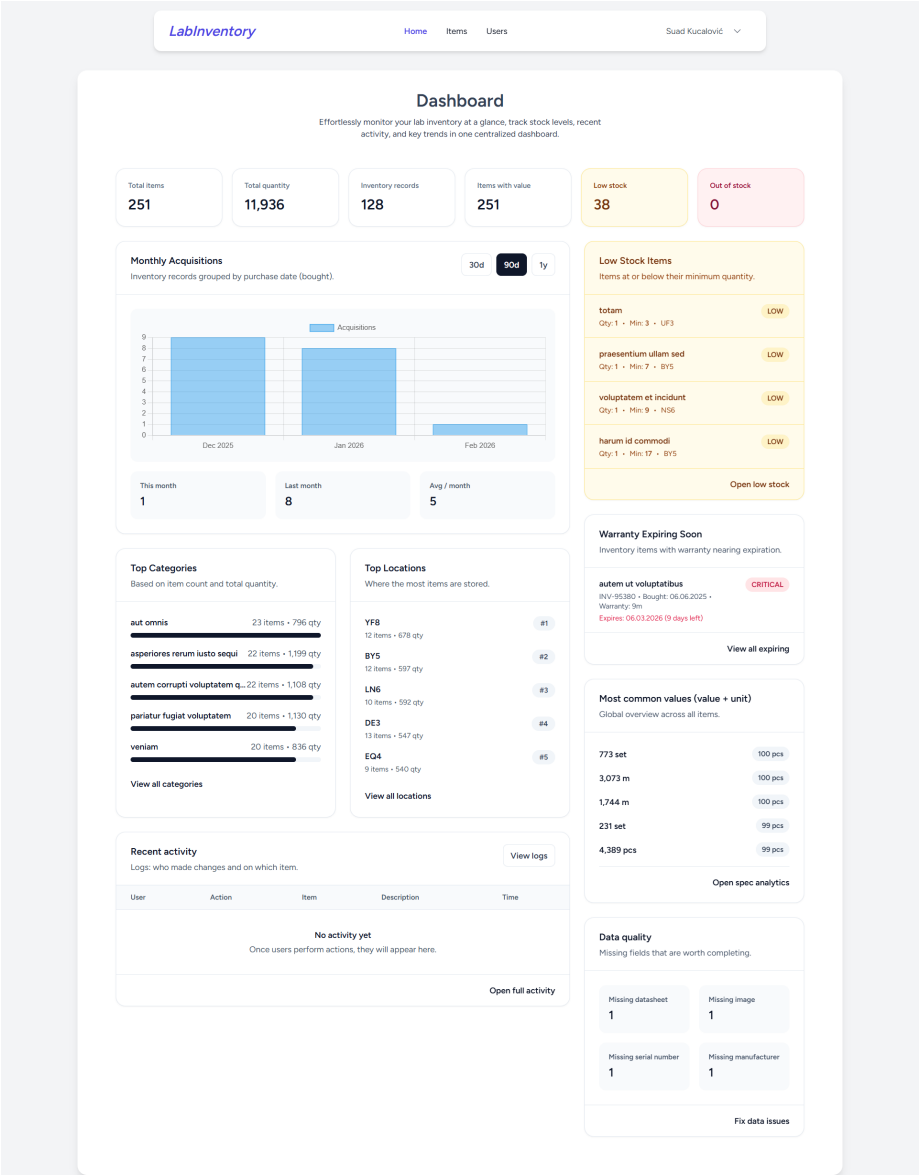


Slika 9: Prijava korisnika na sistem netačna

Slučaj korištenja 2: Učitavanje Dashboard stranice

- Naziv SK: Učitavanje Dashboarda,
- Akteri SK: Prijavljeni korisnik,
- Učesnici SK: Korisnik i sistem,
- Preduslov: Korisnik je uspješno prijavljen,
- Osnovni scenarij SK:

  - Sistem dohvaća statističke podatke iz baze (ukupan broj stavki, količine, low stock, itd.),
  - Sistem generiše grafički prikaz podataka (mjesečne akvizicije),
  - Sistem prikazuje Dashboard sa svim relevantnim informacijama.

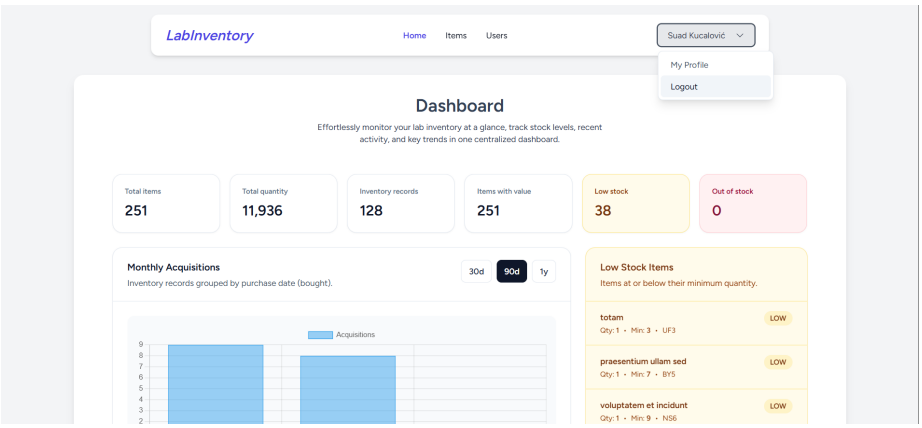


Slika 10: Prikaz Dashboard-a (stranica sa svim relevantnim informacijama)

Slučaj korištenja 3: Odjava korisnika

- Naziv SK: Odjava sa sistema,
- Akteri SK: Prijavljeni korisnik,
- Učesnici SK: Korisnik i sistem,

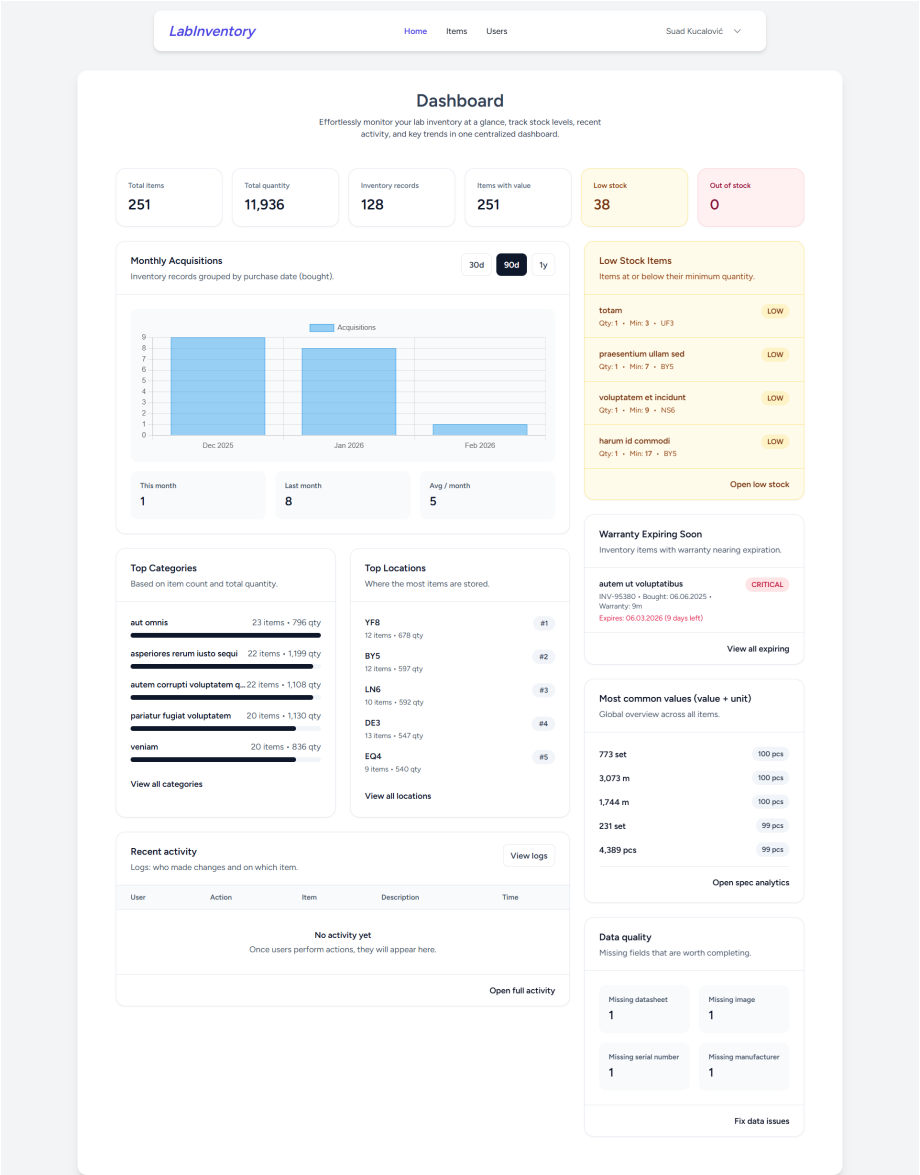
- Preduslov: Korisnik je prijavljen na sistem,
- Osnovni scenarij SK:
  - Korisnik otvara korisnički meni,
  - Korisnik bira opciju *Logout*,
  - Sistem briše korisničku sesiju,
  - Sistem preusmjerava korisnika na login stranicu.



Slika 11: Odjava korisnika sa sistema

Slučaj korištenja 4: Pregled Dashboarda

- Naziv SK: Pregled Dashboarda,
- Akteri SK: Administrator, Laborant, Read Only korisnik,
- Učesnici SK: Korisnik i sistem,
- Preduslov: Korisnik je prijavljen,
- Osnovni scenarij SK:
  - Korisnik pristupa Dashboard stranici,
  - Sistem prikazuje pregled statistika,
  - Korisnik pregleda podatke o stanju zaliha,
  - Korisnik može kliknuti na detaljne sekcije (low stock, warranty, logs).

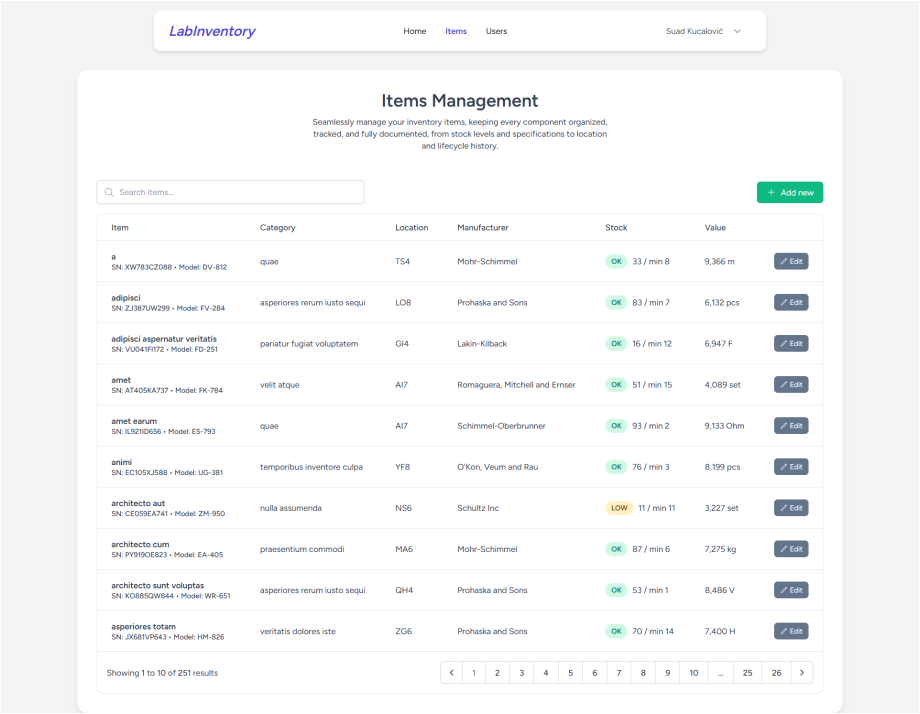


Slika 12: Pregled Dashboard stranice

Slučaj korištenja 5: Pregled stavki (Items)

- Naziv SK: Pregled stavki,
- Akteri SK: Administrator, Laborant, Read Only korisnik,
- Učesnici SK: Korisnik i sistem,

- Preduslov: Korisnik je prijavljen,
- Osnovni scenarij SK:
  - Korisnik otvara sekciju *Items*,
  - Sistem dohvaća listu stavki iz baze,
  - Sistem prikazuje tabelarni prikaz stavki,
  - Korisnik može koristiti opciju pretrage ili filtriranja,
  - Sistem filtrira podatke prema unesenim kriterijima.

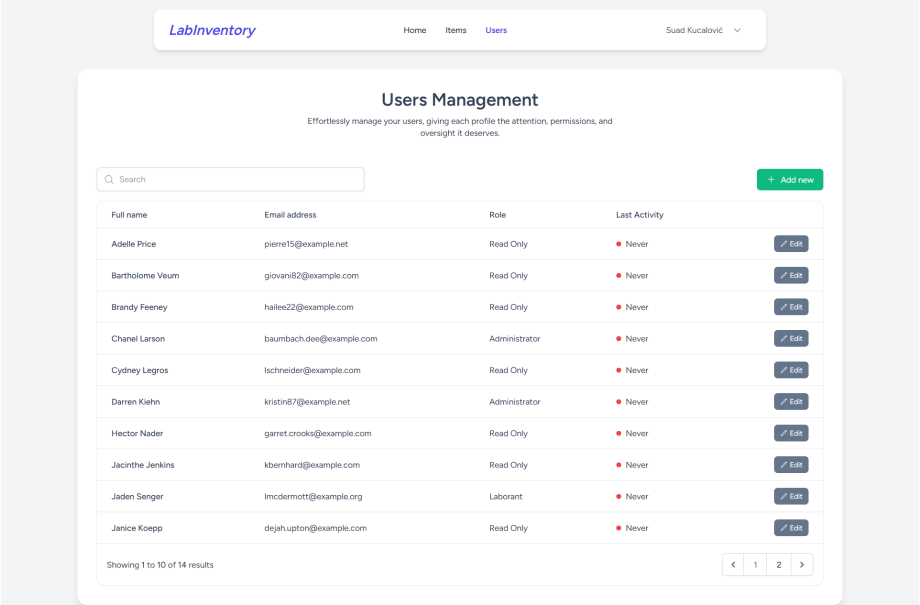


Slika 13: Pregled liste stavki

Slučaj korištenja 6: Pregled korisnika

- Naziv SK: Pregled korisnika,
- Akteri SK: Administrator,
- Učesnici SK: Administrator i sistem,
- Preduslov: Administrator je prijavljen,

- Osnovni scenarij SK:
  - Administrator otvara sekciju *Users*,
  - Sistem dohvaća listu svih korisnika,
  - Sistem prikazuje korisnike sa njihovim ulogama i statusom aktivnosti.

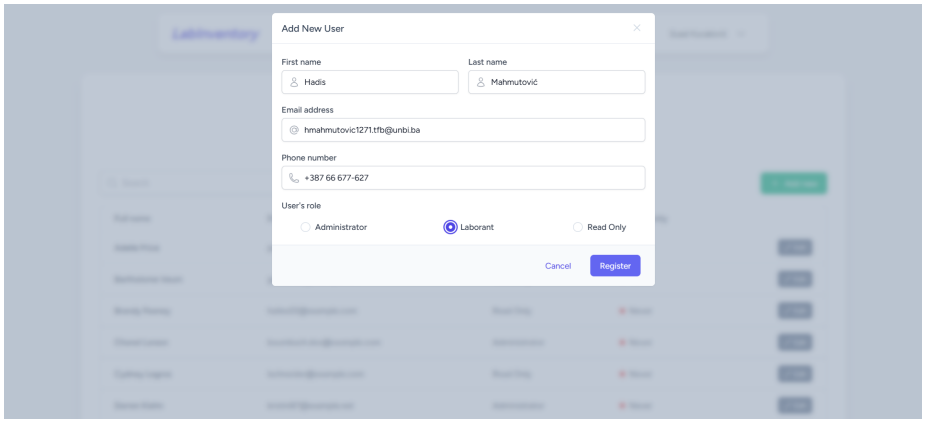


Slika 14: Pregled liste korisnika

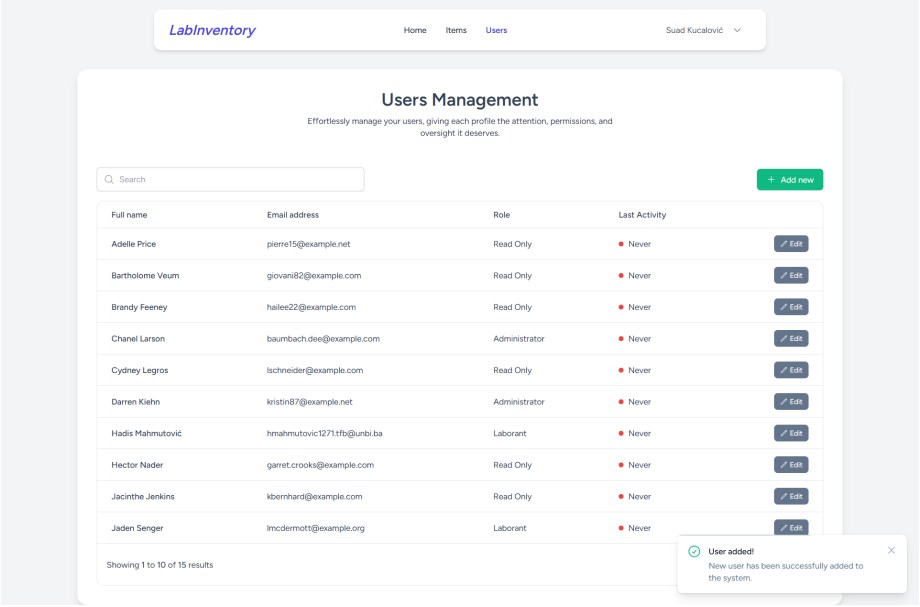
Slučaj korištenja 7: Dodavanje novog korisnika

- Naziv SK: Dodavanje korisnika,
- Akteri SK: Administrator,
- Učesnici SK: Administrator i sistem,
- Preduslov: Administrator je prijavljen,
- Osnovni scenarij SK:
  - Administrator klikne na dugme *Add new*,
  - Sistem prikazuje formu za unos podataka,
  - Administrator unosi podatke i bira ulogu,
  - Administrator potvrđuje unos,

- Sistem validira podatke,
- Sistem kreira novog korisnika,
- Sistem prikazuje poruku o uspješnom dodavanju korisnika.



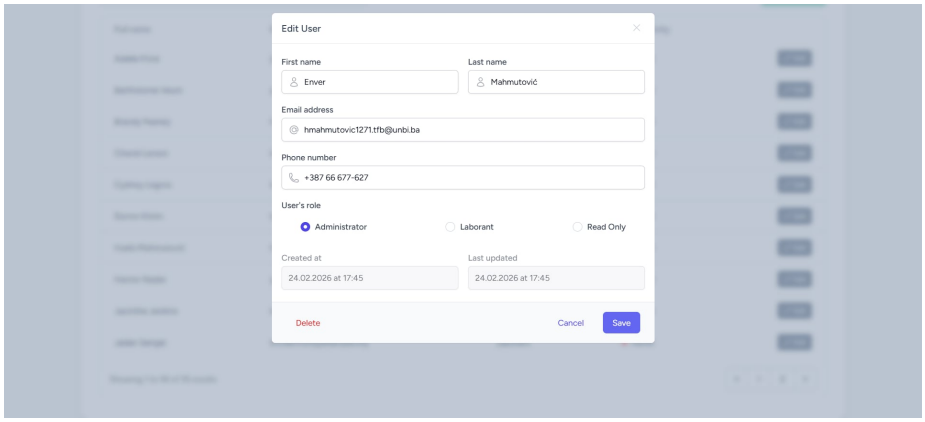
Slika 15: Dodavanje novog korisnika



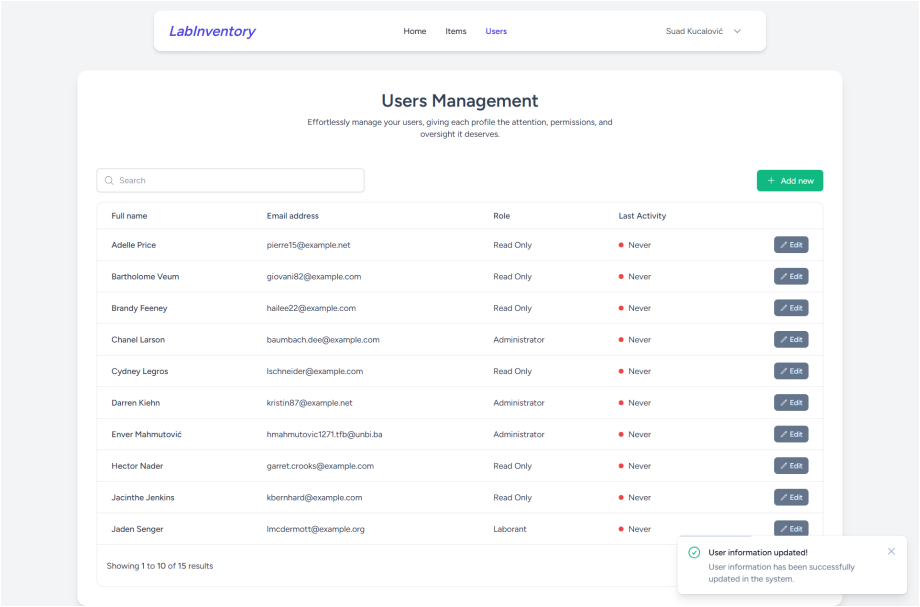
Slika 16: Poruka o uspješnom dodavanju korisnika

Slučaj korištenja 8: Uređivanje korisnika

- Naziv SK: Uređivanje korisnika,
- Akteri SK: Administrator,
- Učesnici SK: Administrator i sistem,
- Preduslov: Korisnik postoji u sistemu,
- Osnovni scenarij SK:
  - Administrator klikne na opciju *Edit*,
  - Sistem prikazuje postojeće podatke korisnika,
  - Administrator mijenja podatke ili ulogu,
  - Administrator potvrđuje izmjene,
  - Sistem validira podatke,
  - Sistem ažurira podatke u bazi,
  - Sistem prikazuje poruku o uspješnoj izmjeni.



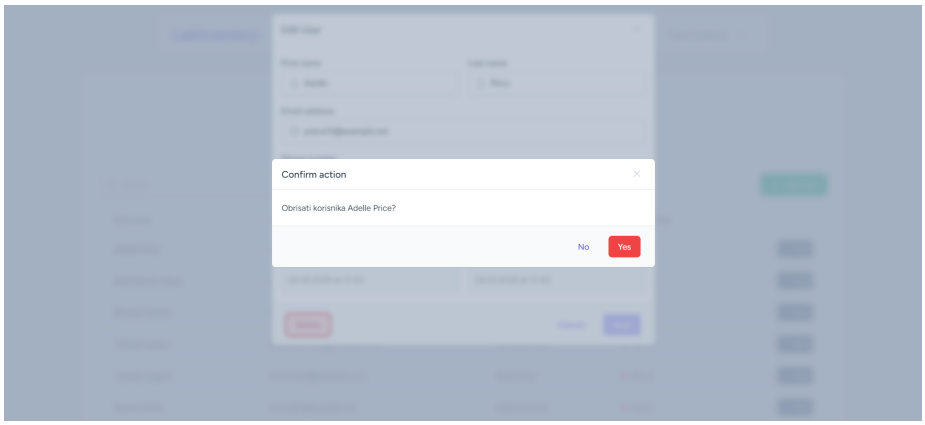
Slika 17: Ažuriranje korisnika



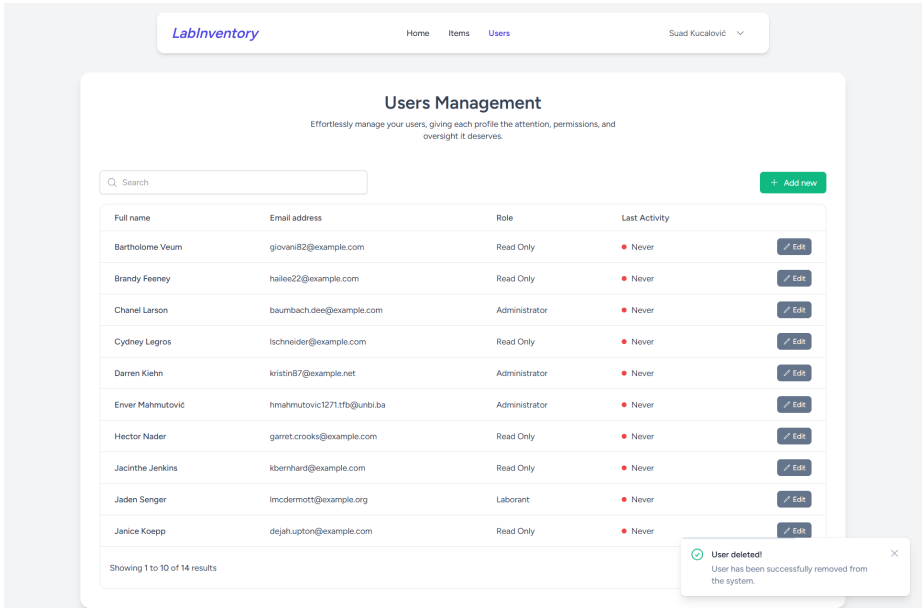
Slika 18: Poruka o uspješnom ažuriranju korisničkih podataka

Slučaj korištenja 9: Brisanje korisnika

- Naziv SK: Brisanje korisnika,
- Akteri SK: Administrator,
- Učesnici SK: Administrator i sistem,
- Preduslov: Korisnik postoji u sistemu,
- Osnovni scenarij SK:
  - Administrator bira opciju brisanja korisnika,
  - Sistem prikazuje potvrdu akcije,
  - Administrator potvrđuje brisanje,
  - Sistem briše korisnika iz baze,
  - Sistem prikazuje poruku o uspješnom brisanju.
- Sporedni scenarij SK:
  - Administrator odustane od brisanja,
  - Sistem zatvara dijalog bez izmjena.



Slika 19: Brisanje korisnika



Slika 20: Poruka o uspješnom brisanju korisnika

4.2 Testiranje API-ja

U ovom dijelu rada opisan je način testiranja REST API-ja aplikacije *LabInventory*. Za testiranje je korišten alat *Postman*, koji omogućava jednostavno slanje HTTP zahtjeva i pregled odgovora API-ja u JSON formatu.

Tokom testiranja provjeravana je ispravnost rada API rute za upravljanje stavkama (*Items*). Poseban fokus stavljen je na provjeru HTTP metoda (GET, POST, PUT, DELETE), odgovarajućih status kodova, kao i validnost i strukturu vraćenih podataka.

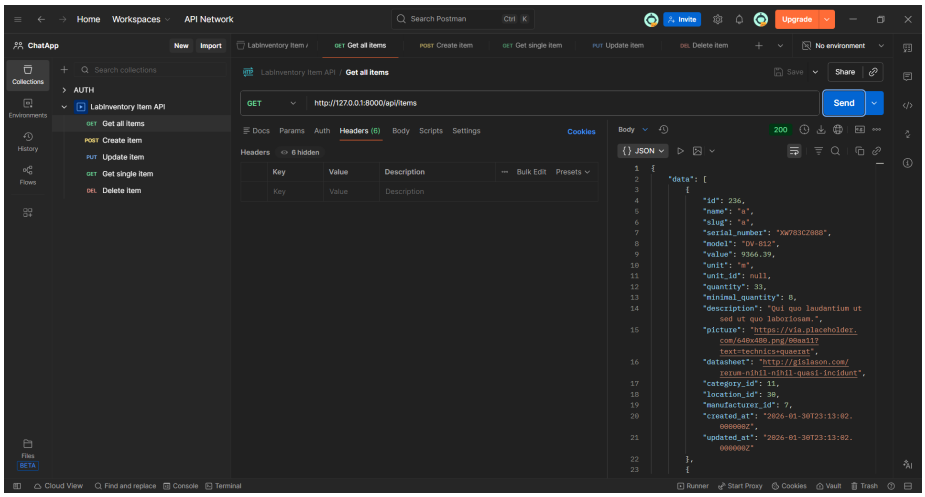
Na slici 21 prikazan je primjer testiranja rute *GET /api/items*, kojom se dohvaća lista svih stavki iz baze podataka. Nakon slanja zahtjeva, server vraća HTTP status kod **200 OK** te JSON odgovor koji sadrži niz objekata sa podacima o stavkama, uključujući identifikator, naziv, serijski broj, količinu, minimalnu količinu, kategoriju, lokaciju i ostale relevantne atribute.

Pored rute za dohvaćanje svih stavki, testirane su i sljedeće funkcionalnosti:

- **POST /api/items** – kreiranje nove stavke,
- **GET /api/items/{id}** – dohvaćanje pojedinačne stavke,
- **PUT /api/items/{id}** – ažuriranje postojeće stavke,
- **DELETE /api/items/{id}** – brisanje stavke.

Tokom testiranja provjeravana je validacija unosa podataka prilikom kreiranja i ažuriranja stavke. U slučaju neispravnog ili nepotpunog unosa, sistem vraća odgovarajući HTTP status kod (npr. 422 Unprocessable Entity) zajedno sa opisom greške u JSON formatu. U slučaju uspješne operacije, sistem vraća potvrdu sa odgovarajućim status kodom (200 ili 201), čime se potvrđuje ispravnost izvršene akcije.

Testiranje je izvršeno u lokalnom razvojnom okruženju korištenjem adrese *http://127.0.0.1:8000/api/items*. Rezultati testiranja potvrđuju da API ispravno obrađuje zahtjeve, validira podatke i vraća konzistentne odgovore u skladu sa REST principima.



Slika 21: Testiranje Items API rute u alatu Postman

5 Zaključak

Razvoj web aplikacije *LabInventory*, namijenjene upravljanju laboratorijskim inventarom, predstavlja uspješnu primjenu savremenih web tehnologija u cilju digitalizacije evidencije opreme i materijala. Aplikacija omogućava centralizovano upravljanje laboratorijskim stavkama, praćenje količina i minimalnih zaliha, evidenciju inventarskih zapisa, kao i upravljanje korisnicima sa različitim nivoima pristupa. Na taj način obezbijeđena je jasna podjela funkcionalnosti prema korisničkim ulogama (administrator, laborant i read-only korisnik), čime je unaprijeđena sigurnost i organizacija sistema.

Aplikacija je razvijena korištenjem Laravel PHP frameworka, koji omogućava organizovan i strukturiran razvoj web sistema kroz MVC (Model–View–Controller) arhitekturu. Ovakav pristup omogućava jasnu podjelu odgovornosti između poslovne logike, prikaza i upravljanja podacima, što značajno olakšava održavanje i daljnje proširenje aplikacije. Korištenje MySQL relacione baze podataka omogućilo je pouzdano i efikasno upravljanje podacima, dok je Eloquent ORM pojednostavio rad sa bazom kroz objektno-orijentisani pristup i implementaciju relacija između entiteta.

U okviru razvoja sistema korišteni su UML dijagrami za modeliranje aplikacije, uključujući dijagrame klasa, slučajeva korištenja, sekvencijalne dijagrame i ER dijagram baze podataka. Ovi dijagrami omogućili su jasnu vizualizaciju strukture sistema, odnosa između entiteta i tokova interakcije između korisnika, aplikacije i baze podataka. Implementiran je i REST API za upravljanje stavkama inventara, čime je dodatno naglašena modularnost i mogućnost buduće integracije sa drugim sistemima.

Testiranje aplikacije obuhvatilo je provjeru ključnih funkcionalnosti, kao što su autentifikacija korisnika, upravljanje stavkama inventara, upravljanje korisnicima i validacija podataka. Korištenjem alata poput Postmana omogućeno je testiranje API ruta i provjera ispravnosti komunikacije između klijentske i serverske strane sistema.

Na osnovu realizovanog projekta može se zaključiti da Laravel i MySQL predstavljaju stabilno i efikasno rješenje za razvoj web aplikacija srednje složenosti, kao što je *LabInventory*. Implementirani sistem pokazuje kako se savremene web tehnologije mogu koristiti za izgradnju pregledne, funkcionalne i skalabilne platforme za upravljanje laboratorijskim inventarom. Projekat pruža dobru osnovu za buduća unapređenja, poput implementacije notifikacija za minimalne zalihe, generisanja izvještaja ili integracije sa sistemima za automatsko označavanje opreme (QR ili bar kod), čime bi se dodatno unaprijedila funkcionalnost i praktična vrijednost aplikacije.

Literatura

[1] Tutorials Point (Model-View-Controller), Dostupno na linku: https://www.tutorialspoint.com/mvc/framework/mvc/framework/introduction.htm/

[2] Laravel , Dostupno na linku: https://laravel.com/

[3] GeeksforGeeks, Dostupno na linku: https://www.geeksforgeeks.org/unifiedmodeling-language-uml-introduction/